Test 1 review day

CMSE 381

Prof. Mengsen Zhang

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Fri, April 18, 2025

Announcements

Announcements:

- midterm # 3 on Monday
- No office hour on Monday, Last office hour on Wednesday.
- Final Project submission on D2L. Due April 25.
- Student Perception of Learning Survey (SPLS): Please fill out this

Based on the questions submitted

- Cubic spline
- Difference between SVC and SVM
- Neural network stuff

Section 1

Cubic Splines

Piecewise polynomials

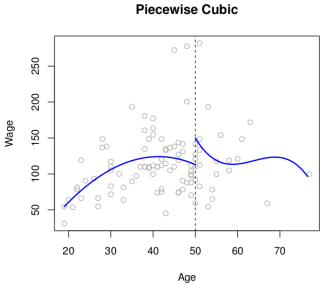
- Fit a polynomial regression

$$y_i = \beta_0 + \beta_1 x_1 + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i$$

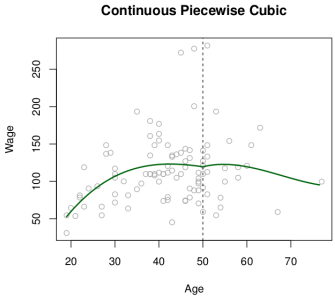
- Let the β_i's be different at different locations of the range.

- Points where the coefficients change are called "knots"
- A piecewise cubic with no knots is just standard cubic polynomial
- More knots gives more flexible polynomial
- Piecewise constant of previous section is degree 0 version of this

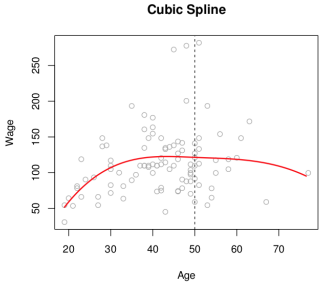
Building up to cubic splines



Piecewise cubic but discontinuous



Continuous but pointy join



Smooth join. Same value plus same first and second derivatives.

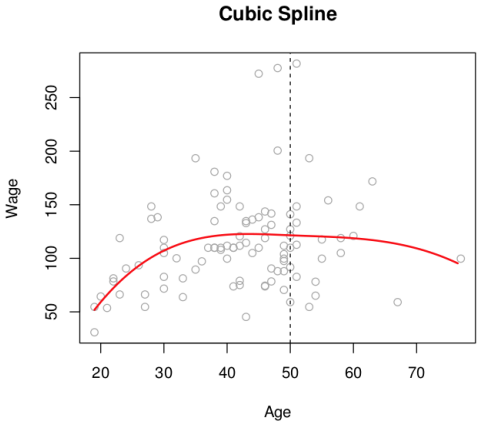
$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \epsilon_i & \text{if } x_i < c \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \epsilon_i & \text{if } x_i \geq c. \end{cases}$$

Test your understanding: PollEv

Cubic splines: degrees of freedom

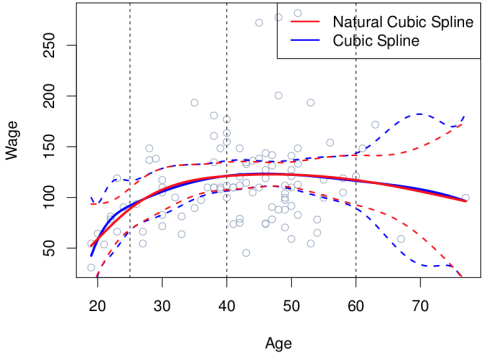
$$f(x) = \begin{cases} \beta_0^1 + \beta_1^1 x + \beta_2^1 x^2 + \beta_3^1 x^3 & x < c \\ \beta_0^2 + \beta_1^2 x + \beta_2^2 x^2 + \beta_3^2 x^3 & x > c \end{cases}$$

- 8 original degrees of freedom, plus three constraints (continuous, 1st deriv continuous, 2nd deriv continuous) leaves 5 degrees of freedom
- cubic spline with K knots uses $4 + K$ degrees of freedom

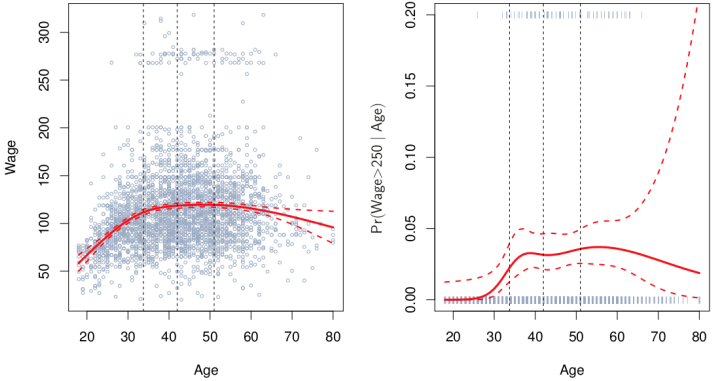


Notes on cubic splines

- Human eyes can't detect the discontinuities so often cubic splines are a good way to go
- Still have problem of high variance at the outer range of predictors (big or small X)
- *Natural spline:* regression spline plus boundary constraints where function is required to be linear at the boundary



Where to put the knots?

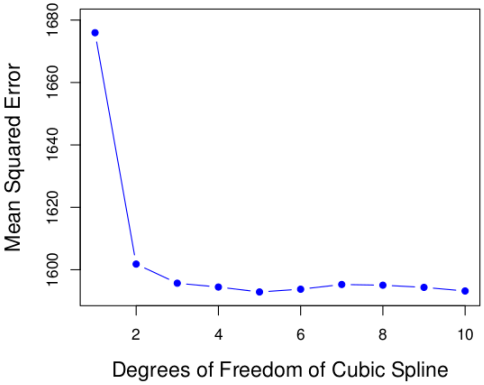


- Plan A: Uniform placement
- Plan B: Data driven. Example here puts it at quantiles of the data

How many knots to use?

When in doubt, Cross-Validate.

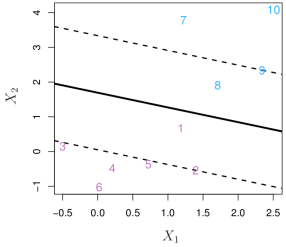
- need to know knot is a flexibility parameter, hence all the bias-variance trade-off stuff applies.
- Do CV (k-fold probably) on data for each number of knots
- Figure at right is the MSE resulting from 10-fold CV
- Note that clearly linear is not enough, after that 3-4 degrees of freedom for the cubic spline are enough

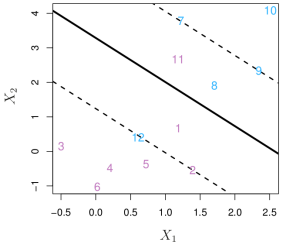


Section 2

SVC vs SVM

Support Vector Classifier





$$\underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n,\,M}{\text{maximize}} \quad M$$

$$\text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M(1 - \epsilon_i),$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^{n} \epsilon_i \leq C,$$

- Soft margin
- Allow for violations across margin
- Allow for violations across hyperplane

So many variables

$$\underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n,\,M}{\text{maximize}} \quad M$$

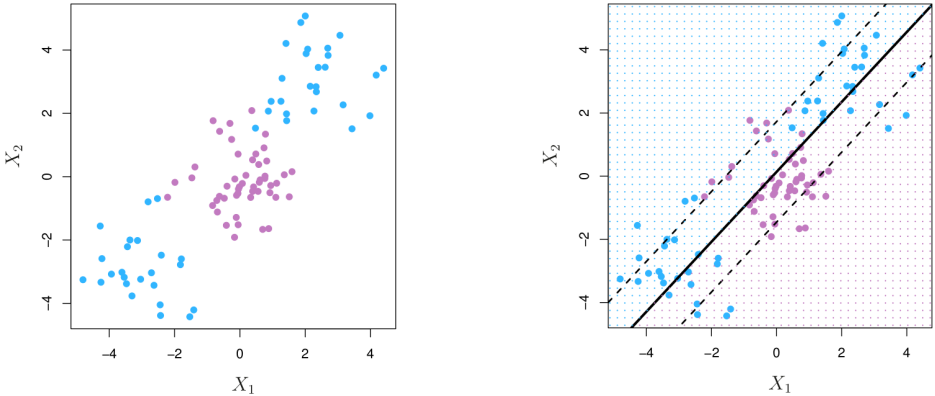
$$\text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M(1 - \epsilon_i),$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^{n} \epsilon_i \leq C,$$

- C is nonnegative tuning parameter
  - Bounds sum of ε_i; number & severity of violating margin (budget)
  - $C = 0$ means no violations allowed
  - $C > 0$ means at most C observations can be on wrong side of hyperplane
- M is the width of the margin
- $\varepsilon_1, \cdots, \varepsilon_n$ are slack variables allowing observations to go to the other side
  - If $\varepsilon_i = 0$, then on correct side of margin
  - If $\varepsilon_i > 0$ then on the wrong side of margin (Violated margin)
  - If $\varepsilon_i > 1$ then on the wrong side of hyperplane

Limiting factor of SVC



- Requires linear boundaries
- "Non-linear" is a lot of things, how do you choose features to learn?
- Right side is result from SVC

More general way of thinking about constraints – Kernels

$$y_i f(x_i) \geq M(1 - \epsilon_i)$$

- Main idea: we can change up $f(x_i)$ to be nonlinear
- In fact, we do not need absolute coordinates of x_i. just need to know how close or similar to each other (draw points w/o coordinates)
- turns out just need to know general similarity between every two points - more computationally efficient
- Kernel: a more generalized way of thinking about how close or similar two samples are
- draw some examples in 1D

$f(x_i)$ should give you some more general way of talking about how far is point x_i away from the decision boundary.

- alternatively, we can simply write $f(K(x_i, x1), K(x_i, x2), \cdots, K(x_i, x_p))$
- this is essentially in relative coordinates

The kernel

$$K(x_i, x_i')$$

- Swap out my inner product $\langle x, x_i \rangle$ for something potentially more complicated
- $\langle x, x_i \rangle$ is known as a linear kernel

Test your understanding: PollEv

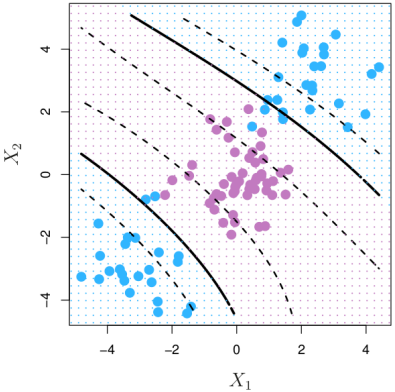
$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i)$$

- The function defined is as above with whatever choice of K
- $K(x_i, x_i')$ can be thought of as similarity function.
- When the support vector classifier is combined with a non-linear kernel the resulting classifier is known as a support vector machine.

A polynomial kernel

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^{p} x_{ij} x_{i'j}\right)^d$$

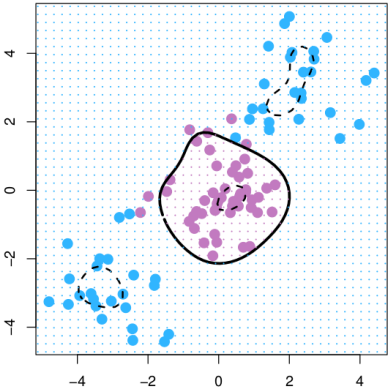
- much more flexible decision boundary.
- amounts to fitting a support vector classifier in a higher-dimensional space involving polynomials of degree d, rather than in the original feature space.
- Right: degree 3 polynomial kernel



A radial kernel

$$K(x_i, x_i') = \exp\left(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2\right)$$

- this is essentially a gaussian kernel
- big distance makes $\sum (x_{ij} - x_{i'j})^2$ big, but then e^{-big} is tiny.
- In $f(x)$, this means that x_i plays little role if it's far away.
- Radial kernel has local behavior



TL;DR

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i)$$

Kernels

- Linear

$$K(x_i, x_{i'}) = \sum_{j=1}^{p} x_{ij} x_{i'j}$$

- Polynomial

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^{p} x_{ij} x_{i'j}\right)^d$$

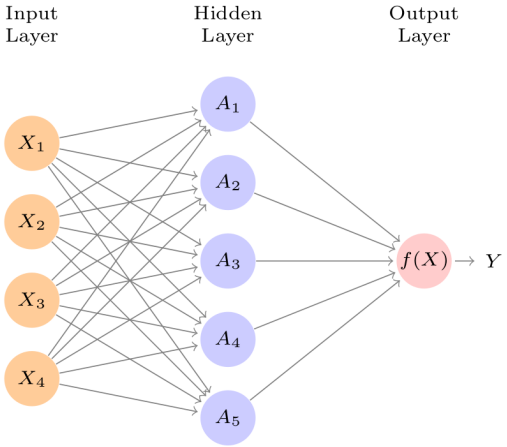
- Radial

$$K(x_i, x_i') = \exp\left(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2\right)$$

Section 3

Neural Nets

Feed Forward Neural Network: The cartoon



$$A_k = h_k(X) = g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j),$$

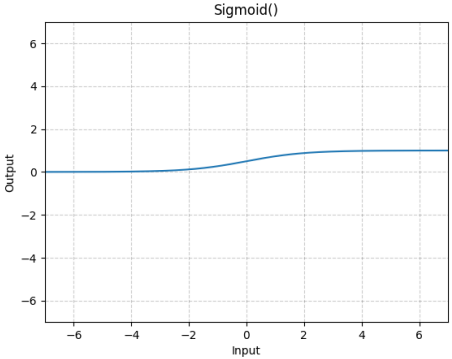
$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

- Input layer is starting data
- Each arrow means taking combo of those values with weights to get next value
- Here there is one hidden layer then the output
- Get to pick how many hidden units K, here we have $K = 5$

Choices for activation function

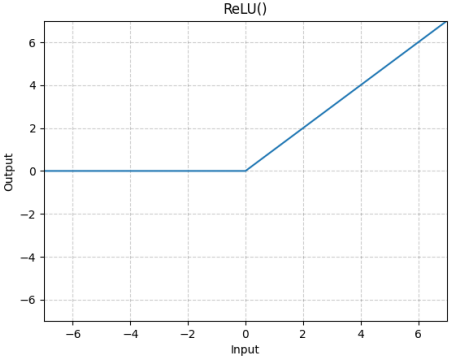
Sigmoid:

$$g(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

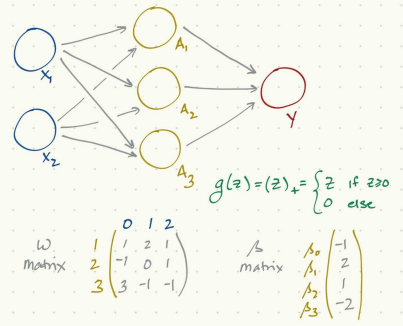


ReLU: Rectified linear unit

$$g(z) = (z)_+ = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{else.} \end{cases}$$



Matrix version



$$A_k = h_k(X) = g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j),$$

$$A = g(\mathbf{W} \cdot \mathbf{X}) \qquad \mathbf{X}^T = (1\ X_1\ X_2\ \cdots\ X_p)$$

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

$$Y = \beta \cdot \mathbf{A} \qquad \mathbf{A}^T = (1\ A_1\ A_2\ \cdots\ A_K)$$